Kubron: план разработки веб-версии и доработки десктопа

Oct 24, 2026 · @Vadim

Цель: из одной кодовой базы собирать два продукта — десктоп (Wails) и веб-сервер (Go + Vue SPA), причём веб должен работать и как «личный» режим, и как многопользовательский сервис для команд DevOps одной организации. Оценка полного пути: 16–20 недель для команды из 3–4 человек (см. «Дорожная карта»). План составлен по анализу репозитория; перед стартом этапа 0 команде нужно сверить номера и списки файлов с актуальной веткой.

1. Цели, границы, допущения

Цели

1. Веб-версия с тем же UI и тем же набором функций, что и десктоп, кроме функций, привязанных к ОС пользователя.
2. Три режима поставки из одного ядра: десктоп; веб «на своей машине» (`127.0.0.1`); веб-сервер для команды или организации (внутри кластера или на отдельной VM/сервисе).
3. Сервисная модель: DevOps разных команд подключают и администрируют свои кластеры, не видя чужих.
4. SSO/OIDC на следующей фазе: архитектура готовится сейчас, реализация позже.
5. Дальнейшая разработка ведётся один раз: новая фича = один метод бэкенда + один вызов на фронте, и она работает в обоих режимах.

Не цели первой версии

- Мобильное приложение.
- Проброс личной идентичности пользователя в Kubernetes (impersonation, его ID-токен): сервер ходит в кластер по его конфигу, права людей задаёт app-RBAC.
- TCP port-forward из браузера (только HTTP через прокси) и бесшовный перенос активных терминалов между репликами: при падении реплики такие сессии открываются заново.
- Миграция на Wails v3 (экспериментальный, строим на v2).

Допущения

- Бэкенд `App` (около 150 методов) остаётся единой точкой входа, и каждый метод первым оператором вызывает `a.authz(...)`; это проверяет `policy_test.go`.
- Фронтенд почти целиком ходит через `callApp(name, ...args)` (`resources/wails.ts`), кроме около 12 файлов с прямыми импортами `wailsjs`.
- Формат JSON-ответов не меняется: Wails сериализует через `encoding/json`, веб-транспорт делает то же.
- Хранилище сейчас — файлы в `~/.kubron` (kubeconfig, `users.json`, токены Argo, `audit.log`).

2. Ключевые архитектурные решения (ADR)

Каждое решение оформляется отдельным файлом `docs/adr/NNNN-*.md` на этапе 0 и утверждается тимлидом до начала реализации.

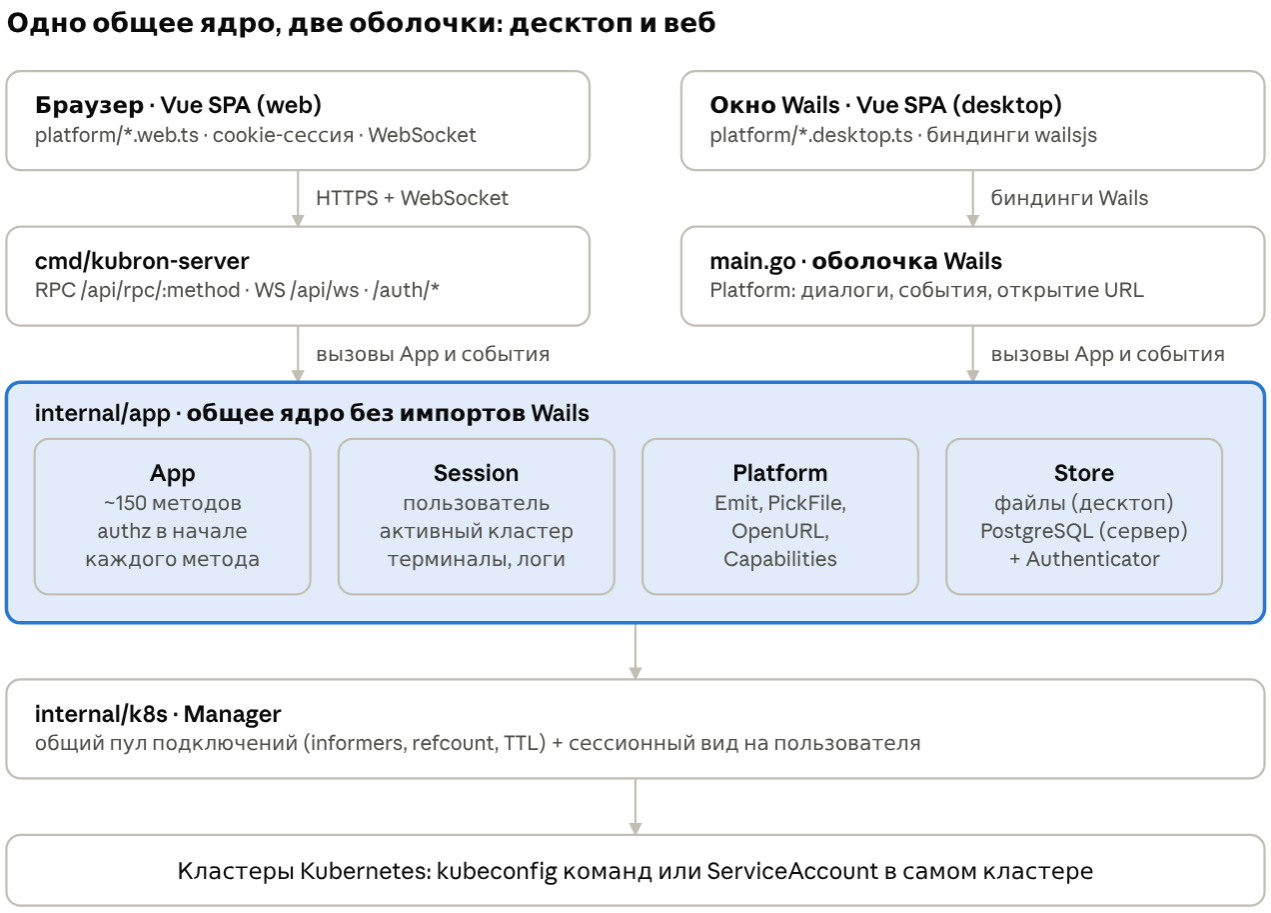
№	Решение	Выбор	Почему и что отвергли
ADR-1	Структура кода	`App` переезжает из `package main` в `internal/app`; две оболочки: `main.go` (Wails) и `cmd/kubron-server` (HTTP)	Второй `main` не может импортировать `package main`. Копировать код отвергнуто
ADR-2	Платформенные возможности	Интерфейс `Platform` (`Emit`, `PickFile`, `OpenURL`, `Capabilities`), две реализации	Убирает `a.ctx` и вызовы Wails runtime из бизнес-логики
ADR-3	Транспорт веба	Универсальный RPC `POST /api/rpc/{method}` (JSON-массив аргументов) + WebSocket для событий	Зеркалит Wails-биндинги, `callApp` почти не меняется. 150 REST-ручек отвергнуты: дорого и дублирует контракт
ADR-4	Сессии	Cookie `HttpOnly; Secure; SameSite=Lax` + CSRF-токен и проверка `Origin`; сессия хранит userID, lastActive, активный кластер	Токены в `localStorage` отвергнуты (XSS). В десктопе одна неявная сессия
ADR-5	Менеджер кластеров	Общий пул подключений (informers с refcount и TTL) + сессионный «вид» с `activeCtx`, терминалами, логами	Не переписывает ~100 мест `GetActiveConnection()`. Текущее «закрыть остальные кластеры при активации» ломает чужие сессии

№	Решение	Выбор	Почему и что отвергли
ADR-6	Модель доступа к Kubernetes	Сервер подключается к кластеру по его конфигу: kubeconfig (токен/сертификат), ServiceAccount или нативный OIDC-вход существующей учётной записью (раздел 7). Права людей — app-RBAC. Проброс личной идентичности в Kubernetes — вне плана	Совпадает с тем, как кластеры уже настроены, и уже заложено в проекте. SSO нужен только для входа в приложение. Риск «сервер = права этой учётной записи для всей команды» снижается ролями приложения и, где возможно, отдельной учётной записью с минимальными правами
ADR-7	Хранилище	Интерфейс `Store` с двумя реализациями: файлы (десктоп) и PostgreSQL (сервер, несколько реплик). SQLite — только для local-режима	Файловые `users.json` и `audit.log` рассчитаны на один процесс. HA нужен, поэтому серверная реализация сразу на PostgreSQL
ADR-8	Секреты	Kubeconfig'и и токены Argo шифруются на диске ключом из env/KMS/Secret (AES-GCM, envelope)	Сервер становится ценной целью, в вебе хранение открытым текстом недопустимо
ADR-9	События	Event hub с адресацией: по сессии, по кластеру, по владельцу ресурса. ID терминала и логов генерирует сервер	Сейчас `EventsEmit` рассылает всем, а ID терминала приходит от клиента: чужое подключение возможно
ADR-10	Фронтенд	Слой `platform/` (`api.ts`, `events.ts`, `files.ts`), реализация выбирается при сборке (`VITE_TARGET=web|desktop`); `GetCapabilities()` скрывает недоступное	Без `if (isDesktop)` по коду
ADR-11	SSO	Стандартный OIDC (authorization code + PKCE) через `coreos/go-oidc`; провайдер за интерфейсом `Authenticator` (локальный логин сейчас, OIDC потом)	Закладываем точку расширения сейчас, реализуем в фазе 2
ADR-12	Мультитенантность	Сущности «организация → команда → кластер», роли назначаются на команду	Требование «DevOps разных команд видят только свои кластеры»

№	Решение	Выбор	Почему и что отвергли
ADR-13	Лицензия и модель	Apache-2.0 для ядра; платные функции — отдельный модуль со своей коммерческой лицензией и лицензионным ключом, подключаемый через интерфейсы (`Authenticator`, `Store`, `Platform`) или build tags. Первый платный модуль — SSO/OIDC: лицензионный ключ с подписью, проверка без сети, срок действия и льготный период. Локальный вход и аварийный админ остаются в ядре	Стандарт экосистемы Kubernetes, минимум барьеров для компаний. Риск: облачный форк ядра без отчислений. Отвергнуты: AGPL-3.0 + CLA (отпугивает часть компаний), source-available (не open source)

3. Целевая архитектура

Одно ядро без зависимости от Wails и две тонкие оболочки: десктопная и HTTP-сервер. Фронтенд тот же, меняется только слой `platform/`.

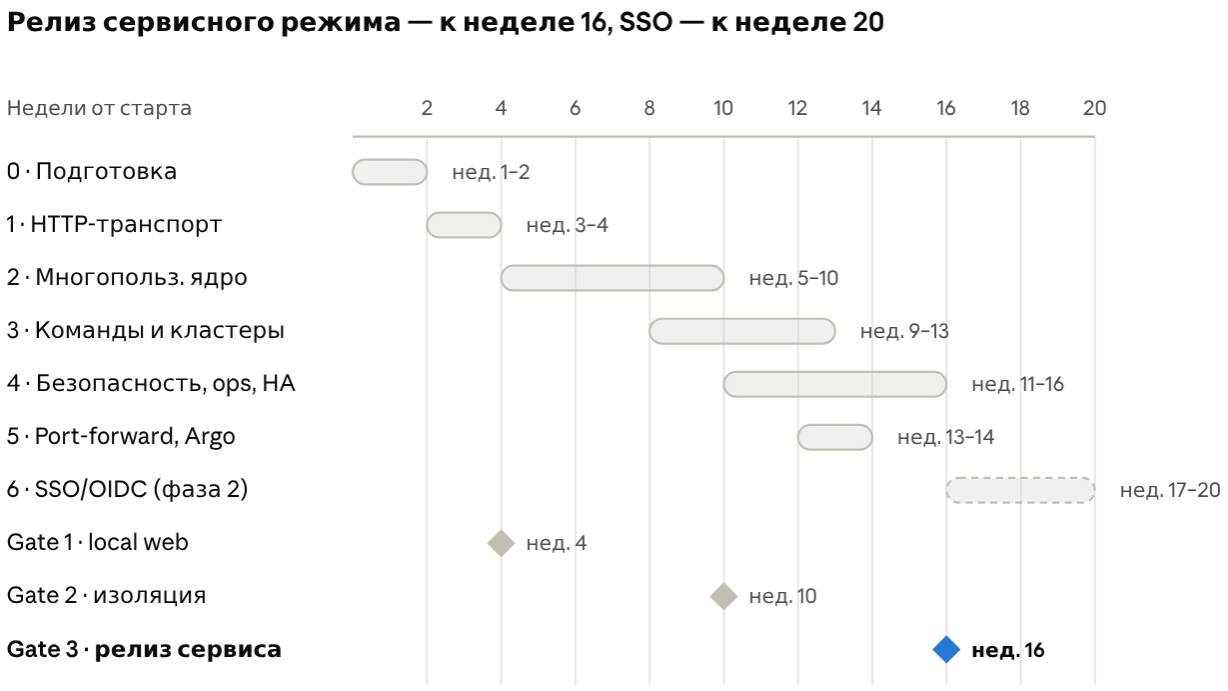


архитектура · 2 оболочки, общее ядро, пул подключений

Ядро `internal/app` получает всё платформенное через `Platform` и `Store`, состояние пользователя — через `Session`. Общий пул подключений в `Manager` позволяет не открывать informers повторно для каждого пользователя.

4. Дорожная карта и этапы с оценками

Релиз сервисного режима — к неделе 16, SSO/OIDC — к неделе 20. Расчёт для команды 3–4 человек, с буфером на review, QA и непредвиденное.



дорожная карта · 7 этапов, 3 гейта, недели 1–20

Этапы 3–5 частично идут параллельно с этапом 2, как только готовы сессии (задачи 2.1–2.2). Гейты — условия перехода:

- **Gate 1.** Веб в режиме одного пользователя проходит e2e (вход, списки, YAML, логи, терминал); десктоп не сломан.
- **Gate 2.** Два пользователя в разных кластерах не влияют друг на друга; `go test -race` чист; нагрузочный тест на 50 сессий пройден.
- **Gate 3.** Раздел 8 выполнен, проведён внутренний security review (или pentest), есть Dockerfile, Helm и восстановление из бэкапа, терминал и HTTP port-forward проходят e2e в вебе; сервис на двух репликах переживает потерю одной.

5. Детальные задачи по этапам

Оценки — в человеко-днях (ч-д) на одного разработчика, знающего код; это приблизительные ориентиры, их нужно уточнить на планировании спринта. Порядок этапов жёсткий только для этапов 0→1→2; этапы 3–5 частично идут параллельно.

Этап 0. Подготовка и разделение ядра (≈ 2 недели, 12–15 ч-д)

Результат: десктоп работает как раньше, но бизнес-логика не знает об интерфейсе Wails.

Задача	Ч-д	Критерий приёмки
0.1 Записать ADR-1…12 (раздел 2) и сверить их с актуальным кодом ветки `refactor`	1	ADR в репозитории, приняты тимлидом
0.2 Перенести `app*.go` в `internal/app`, оставить в `main.go` только Wails-запуск	2	`wails build` и `go test ./...` зелёные; биндинги `wailsjs` без изменений в именах методов
0.3 Ввести интерфейс `Platform`, убрать `a.ctx`, `OpenFileDialog`, `BrowserOpenURL`, `GetFreePort` из ядра в реализацию Wails	3	В `internal/app` нет импортов `wails/v2` (проверяется тестом/`depguard`)
0.4 Обновить `policy_test.go` (сейчас glob по `../../app*.go`) под новый путь	1	Тест проваливается, если метод без `authz`
0.5 Фронт: свести прямые импорты `wailsjs/go/main/App` (~12 файлов: `useK8s`, `useControlledPods`, `actions`, `useTabbedPanel`, `YamlView`, `TerminalView`, `LogsView`, `ResourceEventsTable`, `EndpointSlicesTab`, `PortForwardControl`, `useClusters`, `useAppSettings`) в `platform/api.ts`	3	Прямых импортов нет; ESLint-правило `no-restricted-imports` на биндинги включено в CI
0.6 Фронт: свести `wailsjs/runtime` (~7 файлов: события, открытие URL) в `platform/events.ts` и `platform/files.ts`	2	То же ESLint-правило; десктопный smoke-тест зелёный

Задача	Ч-д	Критерий приёмки
0.7 Добавить `GetCapabilities()` (флаги: `fileDialog`, `portForward`, `nodeShell`, `localKubeconfig`, `sso`) и хук `useCapabilities`	1	В десктопе все флаги true, поведение не изменилось
0.8 Интерфейс `Store` поверх текущих файловых реализаций (`users`, токены Argo, аудит, kubeconfig)	2	Тесты `auth` без изменений логики
0.9 Проверка лицензий зависимостей в CI (go-licenses, license-checker), файл LICENSE (Apache-2.0), заголовки в исходниках, NOTICE, DCO (подпись коммитов) для внешних вкладов	2	CI падает на зависимости с несовместимой лицензией; файл LICENSE и политика вкладов лежат в репозитории
0.10 Публичный пакет расширений (например pkg/ext) с интерфейсами Authenticator, Store, Platform и реестром реализаций: код в internal/ другой Go-модуль импортировать не может	3	Тестовый модуль в отдельном Go-модуле регистрирует свой Authenticator и собирается без правок в ядре

Этап 1. HTTP-транспорт и local web mode (≈ 2 недели, 15–18 ч-д)

Результат: `kubron-server --listen 127.0.0.1:8080` открывает тот же UI в браузере, работает вход, списки, YAML, логи, терминал. Один пользователь; это и есть вариант A из прошлой оценки, точка, где можно продемонстрировать результат.

Задача	Ч-д	Критерий приёмки
1.1 `cmd/kubron-server`: конфиг (флаги + env), graceful shutdown, раздача `frontend/dist` через `go:embed`, SPA-fallback	2	Бинарник один, стартует без Node на машине
1.2 RPC-диспетчер на рефлексии: allowlist методов, разбор аргументов, возврат `(T, error)`, лимит тела, отмена по `ctx` при обрыве соединения	4	Контрактный тест: каждый экспортируемый метод `App` либо в allowlist, либо явно в списке запрещённых

Задача	Ч-д	Критерий приёмки
1.3 Ошибки: единый формат `{code, message}`, маппинг `authz`-отказов в 403, неавторизованного в 401	1	Фронт показывает те же сообщения об ошибках, что в десктопе
1.4 Сессии: логин/логаут, cookie, CSRF, проверка `Origin`, автовыход по `ReportActivity`	3	Нет доступа без cookie; POST без CSRF-токена отклоняется; тесты на idle-timeout
1.5 Event hub + WebSocket (`/api/ws`): подписка/отписка, heartbeat, переподключение с возобновлением подписок	3	`cluster:changed`, `argocd:changed`, `logs:*` доходят только владельцу/подписчику
1.6 Терминал: отдельный binary-WebSocket (`/api/terminal/{id}`) вместо вызова на каждое нажатие клавиши; resize через control-фрейм	3	Задержка ввода не хуже, чем в десктопе на локалхосте ± заметное глазом; чужой терминал по ID недоступен
1.7 Фронт: `platform/api.web.ts`, `events.web.ts`, `files.web.ts` (`<input type=file>` + `AddClusterFromText`), сборка `VITE_TARGET=web`	3	Вход и базовые сценарии проходят в Chromium и Firefox
1.8 Smoke-e2e (Playwright + kind): вход → список pod → YAML → логи → терминал	2	Включён в CI как обязательный job

Этап 2. Многопользовательское ядро (≈ 4–6 недель, 35–50 ч-д)

Результат: два пользователя одновременно работают в разных кластерах на одном сервере и не мешают друг другу. Самый рискованный этап: меняется внутренняя модель состояния.

Задача	Ч-д	Критерий приёмки
2.1 `auth.Service` → общее хранилище пользователей + объект `Session` (id, userID, lastActive, activeCluster); убрать поля `sessionID`, `lastActive` из сервиса	5	Тесты `service_test.go` и `idle_test.go` проходят после адаптации; новые тесты на две параллельные сессии

Задача	Ч-д	Критерий приёмки
2.2 `authz` читает права из сессии запроса, а не из глобального состояния; `App.ForSession(sess)` возвращает лёгкую копию с общими зависимостями	4	В десктопе одна фиксированная сессия; поведение идентично
2.3 `Manager`: выделить общий пул `clusterID → *ClusterConnection` (informers, refcount, TTL без активных подписчиков) и сессионный вид (`activeCtx`)	10	`activateLocked` больше не закрывает подключения других пользователей; тест на утечки goroutine; тест с `-race`
2.4 Терминалы, потоки логов, port-forward — реестр с владельцем (sessionID); лимиты вместо глобальных 50: на сессию и на сервер	4	`onLogout` останавливает только ресурсы своей сессии; ID генерирует сервер; чужой ID → 403
2.5 Адресная рассылка событий: вместо глобального `EventsEmit` — `Emit(target, event, payload)`	3	Тест: пользователь B не получает событий кластера A
2.6 Нагрузка на kube-apiserver: `singleflight` + короткий кэш для `List` на кластер/пространство имён; по возможности перевести частые списки на informer-кэш	6	N пользователей на одном списке = один запрос к API за интервал; нагрузочный тест на 50 сессий
2.7 Переход на `Store` с PostgreSQL для сервера и SQLite для local-режима (пользователи, роли, аудит, метаданные кластеров); миграции схемы; импорт из `~/.kubron`	6	Данные десктопа импортируются командой `kubron-server import`; аудит дописывается атомарно
2.8 Тесты изоляции: два клиента параллельно (Go-тест + Playwright, два контекста браузера)	4	В CI зелёный; `go test -race` без предупреждений

Этап 3. Сервисная модель: организация → команда → кластер (≈ 4–5 недель, 30–40 ч-д)

Подробности — в разделе 6. Кратко: сущности и таблицы, типы подключения к кластеру (раздел 7), управление командами и кластерами в UI, доступ к кластеру по ролям команды, аудит с контекстом команды/кластера.

Этап 4. Безопасность, эксплуатация и HA (≈ 5–6 недель, 50–60 ч-д)

Подробности — в разделах 8 и 9. Кратко: шифрование секретов, rate limit, заголовки безопасности, in-cluster режим, Dockerfile + Helm, метрики и логи, резервное копирование, высокая доступность (несколько реплик, раздел 9).

Этап 5. Port-forward, Argo UI, node shell в вебе (≈ 2 недели, 12–18 ч-д)

Задача	Ч-д	Критерий приёмки
5.1 В веб-режиме `portForward` = true по умолчанию; организация может отключить	1	Порты на хосте сервера не открываются; отключение по настройке организации работает
5.2 HTTP-port-forward через авторизованный reverse proxy `/pf/{id}/...` (сессия и владелец проверяются, таймаут простоя)	6	Чужой поток недоступен; редиректы и абсолютные пути обработаны на примере Argo CD и одного произвольного HTTP-сервиса
5.3 `OpenArgoUI`: в вебе даёт ссылку на реальный URL Argo (если задан) или прокси из 5.2	2	Работает без `BrowserOpenURL(localhost:…)`
5.4 Терминал/exec/node shell: отдельные права в RBAC, аудит начала/конца сессии, лимит длительности, флаг отключения на уровне организации	3	Все интерактивные сессии видны в аудите с пользователем и кластером

Этап 6 (фаза 2). SSO/OIDC для входа в приложение

Подробности — в разделе 7 (≈ 4 недели); реализуется в платном модуле (ADR-13).

6. Сервисный режим: команды, кластеры, изоляция

Сервис для нескольких команд требует трёх новых вещей поверх текущих ролей (admin/operator/user): команды, владение кластерами и область видимости. Предлагаемая модель — минимальная, без сложных политик.

Модель данных

Сущность	Ключевые поля	Примечание
Organization	id, name	В десктопе и local-режиме одна неявная организация и одна команда
Team	id, orgId, name	Единица владения кластерами и видимости
User	id, login, externalId (для OIDC), orgRole, status	`orgRole`: `org-admin` или `member`
Membership	userId, teamId, role	Роль в команде: `admin` / `operator` / `user` (те же права, что сейчас)
Cluster	id, teamId, name, encryptedKubeconfig, mode (`kubeconfig` / `in-cluster` / oidc-password), createdBy	Хранится только в зашифрованном виде
AuditEvent	ts, userId, teamId, clusterId, method, result, ip	Пишется из `authz` на каждый мутирующий вызов

Правила доступа

1. Пользователь видит кластер только если он член команды-владельца.
2. Права решаются в `authz(method, clusterId)`: вычисляется роль пользователя на этом кластере и сверяется с таблицей «метод → минимальная роль», которая уже есть в `policy`.
3. `org-admin` управляет командами и пользователями, но не получает автоматически доступ ко всем кластерам — только к управлению метаданными (принцип минимальных прав).
4. Любой вызов вне списка «мои кластеры» даёт «не найдено», а не «запрещено», чтобы не раскрывать существование чужих кластеров.

Задачи этапа 3

Задача	Ч-д	Критерий приёмки
3.1 Схема БД и миграции для сущностей выше	3	Миграции применяются и откатываются в CI

Задача	Ч-д	Критерий приёмки
3.2 Переработать `authz`: аргумент «кластер», роль через Membership	5	Табличный тест «роль × метод × кластер свой/чужой» покрывает все методы; `policy_test` проверяет, что метод вызывает `authz`
3.3 API управления: команды, участники, кластеры (добавление из текста kubeconfig, проверка соединения, удаление)	5	CRUD доступен только нужным ролям; удаление кластера останавливает informers и чистит секреты
3.4 UI: разделы «Команды» и «Кластеры», переключатель команды, скрытие недоступного по роли	6	В десктопе разделы скрыты через `GetCapabilities` и старый сценарий не меняется
3.5 Аудит: поля team/cluster/ip, фильтры по команде, экспорт	3	Админ команды видит только события своей команды
3.6 Статус соединения кластера и проверка «потерянных» подключений в фоне	3	В UI видно, какой кластер недоступен и почему
3.7 Типы подключения к кластеру: kubeconfig (токен/сертификат), in-cluster, oidc-password (сервер сам получает и обновляет токен); запрет exec в вебе; импорт kubeconfig с oidc-login распознаётся автоматически (спрашиваются только логин и пароль)	6	Кластер с OIDC-конфигом (issuer, client-id, grant password) подключается из веба без kubectl на сервере; токен обновляется до истечения; пароль не возвращается в API и не пишется в логи

Сценарии развёртывания

- **Внутри обслуживаемого кластера.** Режим `in-cluster`: сервер работает под ServiceAccount с настраиваемой `ClusterRole`; kubeconfig не нужен. Рекомендуется давать минимальные права (чтение), а письменные действия включать осознанно.
- **Как сервис «куда угодно».** Сервер в отдельной среде ходит к удалённым кластерам по kubeconfig команд. Требуется сетевая доступность API-серверов от сервера; важны шифрование секретов и политика «кто может добавлять кластеры».
- **Личный режим.** `127.0.0.1`, одна организация, одна команда, без новых требований к инфраструктуре.

Решено: один кластер принадлежит одной команде, поэтому сущность `ClusterGrant` из модели убрана. Если позже понадобится доступ другой команды к чужому кластеру, это отдельная задача вне первой версии. Остаётся выбрать, как сервер подключается к кластеру: см. раздел 7.

7. SSO/OIDC: что заложить сейчас, что сделать позже

Реализация OIDC отложена и войдёт в платный модуль (ADR-13), но четыре решения этапов 0–3 делают её добавление дешёвым (≈ 4 недели вместо переделки ядра).

Закладываем сейчас (внутри этапов 0–3)

Что	Где	Зачем
Интерфейс `Authenticator` (`Login(ctx, credentials) → Identity`) с одной реализацией «local» (логин/пароль, как сейчас)	этап 2.1	OIDC станет второй реализацией, без правок в `authz`
Поля `User.externalId` и `User.provider`	схема БД, этап 3.1	Связь учётной записи с внешним идентификатором; миграция будущих данных не нужна
Таблица «группа IdP → команда + роль» (`GroupMapping`) — пока пустая, с редактором в UI в этапе 6	схема БД, этап 3.1	Роли назначаются группами корпоративного IdP, а не руками
Сессия не зависит от пароля: `Session{userID, authMethod, idpSessionRef?}`	этап 2.1	Нужны повторная проверка и выход через IdP
HTTP-маршруты `/auth/login`, `/auth/callback`, `/auth/logout` выделены отдельно от RPC	этап 1.4	Redirect-поток OIDC не помещается в модель RPC
Вход в UI как компонент с вариантами (`GetCapabilities().authMethods`)	этап 1.7	Кнопка «Войти через …» появится без переделки экрана
Политика «права меняются при следующем входе» + срок жизни сессии в конфиге	этап 2.1	Потеря группы в IdP перестаёт давать права после очередного входа

Делаем в этапе 6 (≈ 4 недели)

1. OIDC: authorization code + PKCE, `state` и `nonce`, валидация ID-токена, поддержка discovery (`.well-known/openid-configuration`). Провайдер не зашит: issuer, client-id, claim логина и групп задаются в конфиге сервера. Тесты на Keycloak и на одном корпоративном IdP клиента (Entra ID, Okta, Google Workspace — по наличию).
2. Роли и команды из claim `groups` по `GroupMapping`; первый вход создаёт пользователя (JIT provisioning).
3. Режимы: «только SSO», «локальный + SSO»; аварийный локальный админ (break-glass), отключаемый конфигом.
4. Выход: локальный + (по желанию) RP-initiated logout.
5. Позже, не в первой поставке: SAML (через промежуточный IdP, например Dex/Keycloak), SCIM-провижининг.

Как сервер подключается к кластеру: отдельно от входа в приложение

Схема «SSO только для входа, а к кластеру — по конфигу» верна и берётся за основу. Вход человека в приложение и подключение приложения к Kubernetes — два независимых слоя. Права человека задаёт роль в команде (app-RBAC), а в кластер сервер ходит под учётными данными самого кластера.

Проблема в вашем конфиге только одна: блок `exec` (`kubectl oidc-login get-token --grant-type=password`). На десктопе он работает как сейчас, так как команда запускается на вашей машине. На сервере не выйдет: нужны `kubectl` и плагин в образе, а режим password у плагина спрашивает логин и пароль в терминале, которого у сервера нет. Кроме того, запуск произвольных команд из загружаемого конфига — риск для сервера (раздел 8). Поэтому в веб-режиме `exec` запрещён, а вместо него приложение умеет четыре типа подключения:

Тип подключения	Как работает	Где подходит
`kubeconfig`	Сервер хранит зашифрованный kubeconfig с токеном или сертификатом	Кластеры без SSO или с техническим токеном
`in-cluster`	ServiceAccount пода с сервером	Сервер внутри обслуживаемого кластера

Тип подключения	Как работает	Где подходит
`oidc-password`	В настройках кластера: issuer URL, client-id, логин и пароль существующей учётной записи (шифруются). Сервер сам получает и обновляет токен и подставляет его как Bearer	Ваш случай: те же issuer, client-id и grant `password`, но без `kubectl` на сервере
`exec` (только десктоп)	Как сейчас: команда запускается на машине пользователя	Десктопная версия и личный веб-режим

Четыре ограничения `oidc-password` нужно учитывать заранее:

1. **Учётка берётся из того, что уже есть.** При добавлении кластера из вашего kubeconfig приложение распознаёт блок `exec` с `oidc-login`, берёт из него issuer, client-id и grant, и спрашивает только логин и пароль. В кластере и в Keycloak ничего менять не нужно. Отдельная техническая учётка — рекомендация, не требование.
2. **Права этой учётки — максимум для всей команды.** Пользователь с ролью `user` в приложении делает то, что разрешает app-RBAC, но от имени учётки, у которой могут быть права админа. Вся защита — роли приложения и его аудит, поэтому проверки `authz` не могут быть неполными.
3. **Жизненный цикл учётки живёт вне приложения:** смена пароля, блокировка, уход человека, чья личная учётка подключена. Приложение показывает владельца учётных данных и состояние подключения («токен не получен», «пароль устарел») и предупреждает админа команды.
4. **Поддерживается только grant `password`.** Кластеры, где в kubeconfig другой grant (интерактивный вход через браузер), в вебе не подключатся без учётки с паролём; на десктопе они работают как сейчас. Grant `password` считается устаревшим в современных рекомендациях OAuth, поэтому тип подключения строится за интерфейсом, чтобы позже добавить другой.

Атрибуция действий по людям в аудите самого кластера не требуется: кто и что сделал, видно в аудите приложения (пользователь, команда, кластер, метод). Проброс личной идентичности в Kubernetes в план не входит.

Платный модуль SSO: сборка и распространение

Рекомендация: платный код живёт в отдельном закрытом репозитории и подключает ядро как зависимость. Ниже сравнение вариантов и шаги; итоговое решение стоит зафиксировать в ADR до этапа 6.

Вариант	Плюсы	Минусы
Отдельный приватный репозиторий (рекомендуется)	Платный код не публикуется; чёткая граница лицензий; сканеры лицензий не путаются	Два репозитория и два CI; ядро должно иметь стабильный публичный интерфейс
Каталог `ee/` в публичном репозитории	Один репозиторий и одна сборка	Платный код виден всем и легко патчится; смесь лицензий в одном репозитории; сложнее принимать внешние вклады
Go plugin или внешний auth-прокси	—	Go plugin хрупкий (та же версия Go, нет Windows); прокси делает SSO решением другого проекта. Отвергнуты

1. **Ядро.** Публичный реестр `authn.Register(name, factory)`; интерфейс `Authenticator` — стабильный API с semver. Только это и нужно заложить до начала этапа 6.
2. **Приватный репозиторий.** Go-модуль импортирует ядро по тегу, в `init()` регистрирует `oidc`; свой `cmd` собирает ядро + OIDC.
3. **Два образа.** Открытый `kubron` (только локальный вход) и закрытый `kubron-ee` (ядро + OIDC). Клиент с лицензией получает доступ к приватному реестру (отдельный отзываемый токен на клиента).
4. **Лицензионный ключ.** Подписанный (Ed25519): клиент, функции, срок. Публичный ключ вшит в платный образ, приватный хранится вне CI. Проверка без сети. После истечения — льготный период с предупреждением в UI, потом SSO отключается, локальный вход остаётся.
5. **Helm-чарт.** Один для обоих образов: `image.repository`, `imagePullSecrets`, `license.secretName`.

6. **CI.** Открытый репозиторий собирает и тестирует ядро, ставит тег. Закрытый ловит тег, собирает платный образ, прогоняет тесты на Keycloak в контейнере, подписывает образ (cosign), публикует в приватный реестр. Версия платной сборки совпадает с версией ядра.

Защита ключа от обхода — договорная, не техническая: Go-бинарник можно пропатчить. Для корпоративных клиентов, для которых важны поддержка и обновления, этого достаточно. Приём платежей и выпуск ключей: на старте вручную скриптом, позже портал; платёжный провайдер зависит от юрисдикции и к коду не относится.

Не забыть:

- [ ] README: честно объяснить, что платная функция — SSO/OIDC, и почему; в ядре остаются локальные пользователи, роли и аудит (смягчает критику «SSO tax»)
- [ ] Не переносить функции из бесплатных в платные после релиза
- [ ] Создать приватный репозиторий и зафиксировать стабильный интерфейс `Authenticator` до начала этапа 6
- [ ] Политика названия и логотипа для форков; проверить, что имя «kubron» свободно
- [ ] Показать лицензию и лицензионный ключ юристу до публичного релиза

8. Безопасность

Веб-сервер с доступом к кластерам — ценная цель: его взлом даёт доступ ко всем подключённым кластерам. Требования ниже — обязательные для выпуска сервисного режима, а не пожелания.

Требование	Реализация	Этап
Транспорт	HTTPS обязателен для не-loopback; TLS на reverse proxy или встроенно через cert-manager/ACME; HSTS	4
Сессии и CSRF	Как в ADR-4; ротация ID сессии при входе; абсолютный и idle-таймаут	1, 2
WebSocket	Проверка `Origin` при подключении, лимит размера сообщения, закрытие при логауте	1

Требование	Реализация	Этап
Защита от перебора	Rate limit по IP и по учётной записи на вход, задержка после ошибок, блокировка с аудитом; хэши паролей — argon2id или bcrypt (проверить текущее хранение)	4
Секреты	Шифрование kubeconfig, токенов Argo и паролей OIDC-учётных записей кластеров (AES-GCM, ключ из окружения/KMS/Kubernetes Secret); секреты никогда не возвращаются в API и не попадают в логи	4
RPC-поверхность	Allowlist методов; методы с файловой системой/диалогами в вебе запрещены; тест на новые методы	1
Пути и SSRF	В веб-режиме нет чтения произвольных локальных путей kubeconfig (в kubeconfig могут быть `exec`-плагины и пути к файлам); запретить `exec`-credential и внешние файлы при добавлении кластера через веб (иначе выполнение команд на сервере); вместо exec — нативный тип oidc-password (раздел 7); проверять адрес API-сервера по политике сети	1, 3
Терминалы и exec	Отдельные права, аудит, лимиты времени и числа сессий; владелец сессии проверяется на каждом кадре	2, 5
Заголовки	CSP (без `unsafe-inline`), `X-Content-Type-Options`, `Referrer-Policy`, `frame-ancestors 'none'`; проверить, что фронт не выводит недоверенный HTML (имена ресурсов, логи, события приходят из кластера)	4
Аудит	Неизменяемая запись мутирующих действий и входов (кто, какой кластер, что, IP); вывод в stdout в JSON для внешнего сборщика	3, 4
Цепочка поставки	`govulncheck`, `npm audit` / `osv-scanner`, подпись образа (cosign), SBOM, скан образа (Trivy) в CI	4
Контейнер	Образ distroless, `runAsNonRoot`, `readOnlyRootFilesystem`, без лишних capabilities; NetworkPolicy в Helm-чарте	4

Модель угроз (сделать на этапе 0)

Короткий документ STRIDE на три границы доверия: браузер ↔ сервер, сервер ↔ kube-apiserver, пользователь A ↔ пользователь B. Только после него можно решать, что именно входит в этап 4. Именно этот документ должен выявить неочевидные дыры вроде `exec` в kubeconfig.

Рекомендация: перед первым внешним развёртыванием провести независимый pentest или хотя бы внутренний review разделов «сессии», «владелец ресурса» и «kubeconfig».

9. Тестирование, CI/CD, поставка

Качество держится на трёх автоматических защитах: контрактные тесты транспорта, тесты изоляции сессий и e2e на настоящем кластере (kind).

Пирамида тестов

Уровень	Что проверяет	Инструмент
Unit	`auth`, `authz`, политика «метод → роль», реестр терминалов/логов, шифрование	`go test -race`
Контракт	Каждый метод `App` в allowlist или в запрещённых; `policy_test` (все методы вызывают `authz`); типы `wailsjs/models` совпадают с тем, что возвращает RPC	Go-тест + генерация в CI
Интеграционные	Сервер + две сессии, общий пул подключений, изоляция событий и терминалов	`httptest` + `envtest` или kind
e2e веб	Вход → кластер → pods → YAML → логи → терминал; два пользователя; тест «нет доступа к чужому кластеру»	Playwright + kind
e2e десктоп	Только smoke (запуск, вход, список); остальное покрыто веб-тестами, так как UI один	Wails + ручной чек-лист на релиз

Уровень	Что проверяет	Инструмент
Нагрузочные	50 сессий × открытый список pod: число запросов к API, память, goroutine	k6 или скрипт на Go

CI/CD

1. Джобы: `lint` (golangci-lint, ESLint, `vue-tsc`), `test-go`, `test-front`, `build-desktop` (Linux/Windows/macOS), `build-server` (бинарник + образ), `e2e-web`, `security`.
2. Генерация `wailsjs` уже есть в CI: добавить проверку «сгенерированное не разошлось с кодом».
3. Релиз: один тег → десктопные артефакты + образ сервера + Helm-чарт; единый номер версии.
4. Версия контракта RPC вшита в сборку фронта; сервер отклоняет запросы со старой версией (кэш в браузере после обновления).

Развёртывание

Вариант	Что делаем
Docker	Один образ, данные в PostgreSQL (volume — только в local-режиме), ключ шифрования из env
Helm (внутри кластера)	Deployment с двумя репликами, PodDisruptionBudget, внешний PostgreSQL, ServiceAccount + RBAC (минимальный по умолчанию), Ingress с TLS, NetworkPolicy, probes `/healthz` и `/readyz`
VM / systemd	Бинарник + юнит, TLS на reverse proxy
Мониторинг	Метрики Prometheus (сессии, подключённые кластеры, ошибки RPC, латентность), структурированные логи (`slog`)
Резервное копирование	Бэкап PostgreSQL + ключ шифрования хранятся отдельно; восстановление проверяется в CI

Высокая доступность (несколько реплик)

Сервис запускается минимум на двух репликах, любая реплика обслуживает любой запрос. Общее состояние живёт в PostgreSQL, интерактивные потоки (терминал, логи, HTTP port-forward) — на той реплике, где они открыты. Дополнительных затрат поверх задачи 2.7 около 25 ч-д.

Тема	Решение	Ч-д
Состояние	Сессии, пользователи, роли, аудит — в PostgreSQL; у реплики нет локального состояния, кроме кэша подключений	6
События	Рассылка между репликами через PostgreSQL `LISTEN/NOTIFY` (в уведомлении только идентификатор, данные читаются из БД); отдельный брокер не нужен. Нужно прямое подключение к БД, не через пулер в режиме transaction	4
Потоки	Реестр потоков с идентификатором реплики-владельца; ingress с привязкой по cookie, чтобы HTTP port-forward попадал на владельца. При падении реплики терминал и логи открываются заново	4
Фоновые задачи	Проверка кластеров, очистка простаивающих сессий, хранение аудита — на одной реплике под `pg_advisory_lock`	2
Подключения к кластерам	Каждая реплика открывает подключения лениво и закрывает по TTL, токены `oidc-password` получает сама. Нагрузка на kube-apiserver и Keycloak растёт пропорционально числу реплик	3
Миграции	Применяет одна реплика под advisory lock; миграции совместимы назад, чтобы обновление реплик по очереди не ломалось	2
Эксплуатация	2+ реплики, PodDisruptionBudget, anti-affinity, `/readyz` проверяет БД, graceful shutdown с кодом переподключения для WebSocket, общий ключ шифрования из Secret	3

Тема	Решение	Ч-д
Проверка	e2e на двух репликах и тест «убить реплику во время сессии»; нагрузочный тест запускается на двух репликах	5

10. Правила разработки двух версий

Работа над двумя версиями добавляет ожидаемо +10–20% к стоимости фичи, если правила ниже закреплены линтерами и тестами, а не договорённостью. Они попадают в `CONTRIBUTING.md` на этапе 0.

1. **Новая фича = метод `App` + вызов `callApp` + (при необходимости) событие.** Без прямых импортов `wailsjs` — закреплено ESLint `no-restricted-imports`.
2. **Метод `App` начинается с `a.authz(...)`.** `policy_test` падает, если нет; тест контракта падает, если метода нет в allowlist/списке запретов.
3. **В `internal/app` и `internal/k8s` нет импортов Wails, глобальных переменных состояния и обращений к `~/.kubron` напрямую.** Всё через `Platform` и `Store`. Проверяется `depguard`.
4. **Платформенные различия только через `GetCapabilities`.** В фронтенде нет `if (isDesktop)` по коду.
5. **Состояние пользователя живёт в `Session`.** Терминалы, логи, port-forward и активный кластер не хранятся в глобальных полях.
6. **Для дорогих точек — кэш и `singleflight` с самого начала.** Новый `List`, который опрашивает API без кэша, не проходит review.
7. **Секреты не возвращаются клиенту и не пишутся в логи.** Для полей с секретами — тег в структуре и тест.
8. **Изменения контракта вводятся обратно совместимо:** новые поля — да, удаление/переименование — через отдельный цикл устаревания.
9. **Definition of Done фичи:** тест бэкенда, ручная проверка в десктопе и в вебе (либо явная пометка, что фича только для одной платформы через capability), запись в аудите для мутирующих действий.
10. **Ветки:** одна основная ветка, фичи — за feature flags, а не долгоживущими ветками; этапы 0–2 не должны блокировать обычную работу над десктопом — поэтому этап 0 делается первым и быстро, одним набором маленьких PR.

11. Риски, команда, открытые вопросы

Главный риск проекта — этап 2 (многопользовательское ядро): меняется состояние, на котором стоит весь код `internal/k8s`. Дальше — риски по убыванию.

Риски

Риск	Вероятность	Влияние	Как снижаем
Рефакторинг `Manager` ломает десктоп	Высокая	Высокое	Держать десктоп в режиме одной сессии, не менять сигнатуры методов, регрессионный набор на kind до начала этапа 2
Утечка данных между пользователями (события, терминалы, informer-кэш)	Средняя	Критическое	Тесты изоляции, адресный `Emit`, review безопасности этапа 2
Нагрузка на kube-apiserver от многих пользователей	Высокая	Среднее	`singleflight`, кэш, informer-чтение, лимиты на сессию и кластер
Сервер с ключами к кластерам становится целью	Средняя	Критическое	Раздел 8, минимальные права, проброс идентичности в фазе 2, pentest
`exec` в kubeconfig даёт RCE на сервере	Высокая, если не запрещено	Критическое	Запрет в веб-режиме + тест на валидацию kubeconfig; вместо exec — тип oidc-password (раздел 7)
Терминал через WebSocket даёт нестабильный опыт за прокси	Средняя	Среднее	Heartbeat, документация по таймаутам ingress

Риск	Вероятность	Влияние	Как снижаем
Растёт объём: «ещё одна фича для веба»	Высокая	Среднее	Замораживать объём этапов, новое — в отдельный бэклог
Отложенный SSO потребует переделки сессий	Низкая	Высокое	Раздел 7: четыре решения закладываются в этапах 0–3
Wails WebSocket/ биндинги меняются в новых версиях	Низкая	Среднее	Десктоп изолирован за `platform/`; замена транспорта — один модуль

Команда и роли

Роль	Загрузка	Отвечает за
Tech lead / архитектор	50% на всём проекте	ADR, модель угроз, review этапов 1–2
Go-разработчик №1	100%	`Platform`, RPC, event hub, сессии, этапы 0–2
Go-разработчик №2	100% с этапа 2	`Manager`/пул, Store/SQLite, команды и кластеры, шифрование
Vue-разработчик	100% этапы 0–3, далее 50%	`platform/`, вход, разделы «Команды/ Кластеры», терминал в вебе
DevOps / QA	50% с этапа 1	CI, kind-e2e, Dockerfile, Helm, метрики, нагрузочные тесты

Если команда меньше (1–2 разработчика), сроки растут примерно линейно: полный путь будет порядка 5–8 месяцев, а первый полезный результат (local web mode) — конец этапа 1.

Открытые вопросы (нужны решения до начала указанного этапа)

- ~~Терминал и port-forward нужны в вебе: решено, этап 5 обязателен; TCP из браузера не нужен, только HTTP через прокси (≈ 2 недели).~~
- ~~Один кластер — одна команда: решено, `ClusterGrant` убран.~~

- [x] ~~Минимум изменений в кластере: решено, подключаемся по существующей учётной записи из kubeconfig (`oidc-password`), техническая учётка — рекомендация.~~
- [x] ~~Атрибуция по людям в аудите кластера не нужна: решено, достаточно аудита приложения.~~
- [x] ~~HA нужен: решено, сервер сразу на PostgreSQL и минимум двух репликах (раздел 9).~~
- [x] ~~Client и realm для входа в приложение: заранее неизвестны, поэтому провайдер не зашивается, всё задаётся в конфиге сервера (раздел 7).~~
- [x] ~~Один IdP на установку: решено. Каждая организация разворачивает свой экземпляр kubron; несколько IdP в одном экземпляре вне плана.~~
- [x] ~~PostgreSQL предоставляет и устанавливает пользователь, kubron его не ставит: решено, Helm-чарт принимает строку подключения из Secret.~~
- [ ] Лицензирование и модель распространения: ядро под Apache-2.0, платные функции по подписке — отдельный модуль с коммерческой лицензией. Платная функция на сегодня только одна: SSO/OIDC (этап 6), остальное — открытое ядро

12. Платный модуль (SSO/OIDC): сборка и распространение

Рекомендация: отдельный приватный репозиторий с Go-модулем, который импортирует открытое ядро, регистрирует свой `Authenticator` и собирает свой образ. Решение нужно принять до этапа 0 из-за задачи 0.10: интерфейсы расширения должны жить в публичном пакете, а не в `internal/`.

Варианты сборки

Вариант	Плюсы	Минусы
A. Отдельный приватный репозиторий (рекомендуется)	Чистое разделение лицензий, платный код не виден публично, нет путаницы у контрибьюторов	Два репозитория и два CI; ядро должно держать стабильный API расширения
B. Каталог `ee/` в публичном репозитории с другой лицензией и build tag	Один репозиторий, один CI (так делает GitLab)	Код платной части легко собрать без ключа и обойти проверку; мешает прозрачности лицензии «Apache-2.0 целиком»

Вариант	Плюсы	Минусы
C. Внешний плагин (отдельный процесс или Go plugin)	Можно ставить платный модуль к готовому образу	Сложность неоправданна для одной функции; Go plugin хрупок к версиям

Распространение

Что	Как
Открытое ядро	Десктопные бинарники в GitHub Releases, образ `kubron` в публичном registry (ghcr.io или Docker Hub), Helm-чарт как OCI-артефакт
Платная сборка	Образ `kubron-enterprise` в приватном registry; подписчику выдаётся токен на чтение образа и лицензионный ключ. Это надмножество ядра: без ключа он ведёт себя как обычное ядро
Версии	Один номер версии. Сначала релиз ядра, потом автоматическая сборка платного образа на этом теге. CI платного репозитория также каждую ночь собирается против ветки `main` ядра, чтобы ломающее изменение расширения было видно сразу
Подпись	Оба образа подписываются (cosign) и имеют SBOM

Лицензионный ключ

1. Подписанный токен (подпись Ed25519): организация, список функций, срок действия. Публичный ключ зашит в платный модуль, приватный хранится у вас офлайн.
2. Проверка без сети, без «звонка домой»: организации часто работают в изолированных сетях. Ключ отзывается только сроком действия (практично — год).
3. Поведение при просрочке: SSO отключается после льготного периода, локальный вход и аварийный админ работают; организация никогда не теряет доступ к своим кластерам.

4. На первом этапе биллинг не нужен: счет вручную + небольшая CLI-команда выпуска ключа. Связка «оплата → ключ» автоматизируется, когда появится первые клиенты (Stripe, Paddle или аналог).
5. Проверка ключа не делает защиту абсолютной: Go-бинарник можно править. Цель — честные клиенты, а не защита от взломщиков; закрытый репозиторий платного кода даёт больше защиты, чем проверка.

Что не забыть (продуктовые и юридические заметки)

- [ ] README: честно объяснить, что SSO платный и почему; в ядре остаётся простой вход (локальные пользователи, роли, аудит). Сообщество часто критикует «SSO tax», поэтому правила должны быть публичны и понятны заранее.
- [ ] Не переносить функцию из бесплатной в платную после выпуска; обратное нормально.
- [ ] Стабильный публичный API расширения (semver для `pkg/ext`): от него зависит платный модуль.
- [ ] Счёт, условия подписки и коммерческая лицензия платного модуля — текст согласовать с юристом.
- [ ] Проверить свободное название и определить политику товарного знака для форков: лицензия Apache-2.0 бренд не защищает.